

UFO 5: Individual Reflection

Michael Hardwicke GAME 1501 Rapid Prototype Development, Assignment 2 Bow Valley College, Summer 2026

Five prototypes in seven weeks, with the group and the constraint both changing between each one. What follows is what actually changed in how I work, anchored to the moments that changed it.

Constraints relocate the design question, they do not shrink it

I went into Ghost expecting “one button” to mean less game. It meant a different game. The move that made the prototype work was treating the constraint as one button *per person*, which put two priests either side of a possessed man and made the whole thing a tug of war. The question stopped being “what does the button do” and became “who is pressing it, and what happens to the loser.” Nothing about the constraint suggested that. It just stopped suggesting anything else.

That cycle carried a second constraint, the theme “I cast,” and the two behaved completely differently. “One button” narrowed the mechanic and opened the fiction. “I cast” did the reverse: it is a pun with a dozen readings, so it opened the fiction and narrowed nothing at all until we forced it to. We spent most of the first day listing what “cast” could mean, fishing and magic and casting a limb and casting a role, and the reason exorcism won was that it was the only one where the pun and the button described the same physical act. Sustained effort against resistance is what mashing feels like. A constraint that only constrains the story is not doing half the work, and I did not know that going in.

The strangest constraint of the term was on UntitledGardenGame, where Du and I had to agree on something we both actually disliked before we were allowed to design anything. That is not a constraint on the artifact at all, it is a constraint on the pair, and it turned out to shape the build more than the rule about stealing all the art did. We spent the first day on wealth disparity, algorithmic curation, speculative bubbles, dragon sickness, and what we converged on was not a topic but a shape: systems that extract from people who cannot opt out. Everything downstream followed from that. The game is about building mutual aid infrastructure rather than fighting in the street, and it ends with a functioning commune rather than a toppled government, because that was the thing we were both angry about. I have never had a brief target the team rather than the product before and I would use it again.

The same thing happened at the other end of the term. “Countdown” reads as a fail timer, and every version of Count Down Under we sketched on the whiteboard on 22 July had the sun killing you outright. The reading we landed on instead was that sunrise starts draining what you have gathered rather than ending you, which turns the clock into a decision about when to stop pushing your luck. That is a stronger pressure than a fail state, and we only found it by asking what the constraint was *for* instead of what it required.

What I did not notice until writing this up is that we then built two different answers to that question, one in the design document and one in the level, and never reconciled them. More on that below.

The one I got least out of is the despised mechanic on Finding_Keys, and it is the only cycle where the constraint was hostile on purpose. We drew “finding keys for lock puzzles” out of a hat and the brief was to make somebody enjoy it. Our answer was to give the keys a target: an alien you herd through a hospital by closing its exits, so a card is not a collectible, it is a move. That is a real answer and it is a partial one.

The mechanic is not despised because keys are boring. It is despised because the search is unfalsifiable. You cannot rule anything out, so you open everything, and in our build a searched cabinet and an unsearched one look the same from across the room. We treated it as a pacing problem and added a chase. It is an information problem and the fix is to let the player deduce where a card is rather than sweep for it. I did not see that during the build. I saw it writing this up, which is the second time in this document I have had to admit that the analysis arrived weeks after the opportunity to act on it.

The evidence beats my theory of the evidence, every time

I lost the most time in this term to being confidently wrong about causes.

On GMTKUFO, a fully drained victim awarded 99 blood instead of 100. I spent real time in the drain code looking for an off by one in the transfer. It was `Mathf.FloorToInt` applied to a float that summed to 199.99998. There was no drain bug. There was a rounding bug wearing a drain bug's clothes.

Worse, the tool I wrote to diagnose the bite failures reported a drained victim as "in range on all axes, so the miss is the layer mask," because `OverlapBox` skips disabled colliders and `Die()` disables the victim's. I built an instrument, trusted it, and it pointed at the wrong subsystem. The same afternoon, the Animator window showed `Drac.controller` correctly bound while the old controller swapping code was the thing actually running, because the file had been renamed and kept its guid. That false confirmation is what made me think a refactor was working when it was not, and it cost most of an afternoon of a four day jam.

It happened once more while writing this assignment, which is why I am confident it is a pattern rather than three bad nights. Reconstructing the Finding Keys cycle, I read Danish's commit "Removed cornered state, finalised code" and wrote up a story where playtesters had abused the box trapping so we cut it. It is a good story. The diff says the opposite: he removed the *accidental* cornered state, where the alien trapped itself by failing to find a path, and made the boxes the only route to a win. I had inverted the entire revision, from a commit message, because the wrong reading was more interesting than the right one. Ten minutes with `git show` would have caught it, and I only ran it because a recorded playtest contradicted me.

The practical change is that I now write down what I measured rather than what I concluded. `DEBUG_NOTES.md` on GMTKUFO has a "Dead end, do not repeat" section and frame counts against specific commits, because the version of me at 2am does not deserve to be trusted about causes. The corollary I have just learned is that a commit message is a conclusion too, and my own six week old summary of my own work is a secondary source.

Playtest evidence works when it is phrased as behaviour

Compare two rounds of feedback on SpinSpinSpin. The 23 June round produced four notes, all of them descriptions of what players did: they lost a run in the first second after unpauseing, one obstacle contact ended the run and nobody understood why, nobody could tell what they had collected, getting hit produced no reaction. All four shipped inside two days, and the fixes were obvious once the behaviour was stated.

The 29 June round included "too fast, too easy, no roughage," which is a judgement rather than a behaviour, and it is the one that took the longest to act on. Not because it was wrong, but because it was not actionable until I worked out what was underneath it. What was underneath it was that only I could place obstacles, so difficulty could only be tuned by whoever held the code. The fix was not a tuning pass, it was `SpawnWave`, a text grid where a wave is painted one character per lane so anybody on the team can author difficulty.

Sean's line on 15 July, that we need to be more critical with feedback because the class is too nice, is the same observation from the other side. Nice feedback is usually feedback phrased as a judgement, because judgements can be softened and behaviours cannot. "Start button hard to click" is not unkind, it is just true, and it got fixed the next day.

I build tools for other people, and I should be more suspicious of that

The pattern is unmissable across five cycles. SpawnWave and its bucket system on SpinSpinSpin. RevTechTreeWindow, RevEndingMapWindow and RevContentGenerator on UntitledGardenGame. LevelPopulator on GMTKUFO. HitFeedback polling the hit count rather than listening to obstacles, so every hazard added later inherits the juice without anybody wiring it. By the last cycle the team gave me the role name "systems," which is accurate.

I have described myself as more of an editor than a creator, and I now think building authoring tools is how I convert creation into editing. I set up a frame and let other people fill it, and then my job is refining what comes back. That is a genuine strength on a team of four with a real content load. It is a liability when it is a reflex.

On UntitledGardenGame it clearly paid: two people, a week, and a content heavy design where Du writing actions and endings in parallel was the difference between shipping and not. On GMTKUFO it is harder to defend. Four days, four people, and a build whose bottleneck was never content volume, it was the player controller not behaving. LevelPopulator was a bet on a problem I did not have. The honest version is that tooling is comfortable for me in a way that debugging a physics shove is not, and I reached for the comfortable work while the actual blocker sat there.

SpinSpinSpin is the case that taught me what the tools are actually for, and I only understood it while writing these notes. Four of us, all with jobs and families, and almost no time in a room together. We worked remotely and steadily, but every decision cost a round trip of messages instead of ten seconds of conversation. When the feedback came back "too fast, too easy, no roughage," I read it as a difficulty problem and built SpawnWave to fix it. It is not a difficulty problem. It is a bandwidth problem. A wave painted as text that anyone on the team can open and edit is a way of making a decision without holding a meeting, and on a team that cannot hold meetings cheaply that is worth a day of a seven day jam.

So the rule is not quite "build the tool when someone is blocked." It is: build the tool when the *conversation* is expensive. On a pair sitting together, a tool is overhead, because talking is free. On four people across four time zones of availability, the tool is the conversation. That reframes the whole habit for me, and it also explains the GMTKUFO misfire, where I built for a content problem on a team whose expensive thing was debugging.

Split freely, rewire never, at least not during a jam

Two refactors, seven days apart, opposite results.

On UntitledGardenGame, "Refactor to reduce file lengths" on 20 July and "Make files less than 300 lines" on 21 July turned RevGameManager into partial classes across Days, News and Queue, and split RevGameScreenController seven ways. Cost: almost nothing. Behaviour unchanged, moving text between files.

On GMTKUFO on 25 July, replacing animator controller swapping with a single parameter driven controller never worked and ate an afternoon. The refactor is still the better design. It needs a branch and testing time.

The difference is not size, it is whether the change alters how a working system works. Splitting is safe under time pressure because the failure mode is a compile error. Rewiring is not, because the failure mode is subtle behavioural drift, and drift is expensive to spot

when you are also the person who introduced it. That distinction was not obvious to me before I had both experiences a week apart.

Rotating groups changed what I write down, and how much

Ghost was a pair, and the only document it needed was a setup guide for wiring buttons into the state machine. Kyle and I could hold the rest in a conversation. GMTKUFO was four people who had not all worked together, over four days, and it produced a schedule, naming conventions, a GDD and a debug log running to tens of thousands of words.

It also produced the least functional build of the five. I want to sit with that rather than explain it away, because the neat version of this reflection would have the documentation and the quality moving together and they moved in opposite directions. `DEBUG_NOTES.md` is not a record of a well run project. It is a record of firefighting, written because things kept breaking in ways I could not hold in my head. The documentation was diagnostic, not preventive, and its length is a symptom.

The same cycle shows the other failure mode. The GDD written on day one describes a run with no fixed ending, where you sleep in a coffin whenever you decide you are done. The level that got built has a start coffin and an end coffin, which is a race. Those are different games and we never chose between them out loud, because for most of four days there was no playable level to check the document against. A design document stops being true the moment nobody is comparing it to a build, and it does not tell you when that has happened. It just keeps sitting there sounding authoritative.

The tempting lesson is that more documentation is better. I do not think that is right. Documentation is a coordination cost, paid to replace conversations that cannot happen. On a pair with adjacent desks it is waste. On four people across a weekend it is the only thing that stops the same bug being investigated twice.

SpinSpinSpin sits in between and is the clearest test of that claim, because there the conversations could not happen for a reason that had nothing to do with the work. Four people with jobs and families and almost no shared in-person time. We shipped, and the build is good, and I still think we left a lot on the table, because a jam runs on the speed of the loop between noticing something and changing it. What I would do differently is not “write more docs.” It is to recognise on day one that the team is bandwidth limited and spend the first day building for asynchronous work, rather than arriving at the same conclusion on day six by accident and calling it a difficulty fix.

What did transfer between every group was convention. The `_Project` folder root with Code split into Core, Gameplay and UI is on every build from Ghost onward, and by the last two cycles I did not have to argue for it, because a repo that looks the same as the last one lets a new collaborator find things without asking. That is worth more than any document, and it costs nothing after the first time.

What I would do differently

Keep structured playtest records at the time. Every revision note in the accompanying document was reconstructed from Sean’s board, my notebook and commit messages. That reconstruction was possible, which says something about commit hygiene, but it is archaeology rather than evidence. A single sheet per session with who played, what they did, and what surprised us would have taken five minutes a round and would have made every claim in this assignment primary.

Front load one hostile playtest per cycle. In each of the five cycles the change that mattered most came from the first time somebody who had not built it tried to play it, and in four of the five that happened later than it needed to. On `Finding_Keys` the alien could

corner itself by failing to find a path, handing the player a win with no keys involved, and that was findable on day two. Danish caught it on day six and rewrote the win condition the same night.

Use the scope document as a cutting tool, not a wish list. I wrote scope statements for this assignment retroactively and several of them were clearer than anything we agreed at the time. The instructive case is GMTKUFO, because there we *did* write a minimum viable product list on day one, and it made no difference at all. We did not hit it. Having the document was not the practice. Returning to the document when the situation changed was the practice, and we never did that once.

The situation had changed before we started: every other cycle in this term was seven days and that one was four. We planned a week of work on a four day calendar and nobody re-opened the plan when the number was different. An MVP list you do not cut against is just a description of what you hoped for.

The thing I did not expect

Sitting in the last session on 25 July, dividing roles for a group I had partly worked with before, I noticed that I do better work with people nearby and options in front of me than I do alone with a blank file. Five rotating groups is an unusual amount of evidence for that, and it is not what I would have predicted about myself at the start of the term.

SpinSpinSpin is the proof, and it is proof by absence. That team was capable and the idea was strong and we could almost never be in a room together, and the thing that suffered was not any individual's output but the speed of every decision the four of us had to make jointly. I spent that jam building a way to work around not being able to talk to my team, and I did not notice that was what I was doing until I wrote it down six weeks later.

The constraint rotation taught me about design. The group rotation taught me something more useful and less comfortable: that proximity is a resource I depend on more than most, that its absence shows up as a slow team rather than an unhappy one, and that I should be planning for it at the start of a cycle rather than compensating for it at the end.

What I am taking into the next build

GMTKUFO carries forward as the Assignment 3 project, so this reflection has somewhere to land rather than being filed.

The tempting story is that four days was the problem and more time will fix it. I do not believe that. We committed on day one to a 3D controller with a state machine, two player forms with an animator swap, three hazard types, a dynamic sun casting real shadows, shadow detection on the player, and a resource that is score and health at once. That is a seven day plan, and we were given four, and the plan never changed. More time applied to the same overreach produces a slightly less broken version of the same thing.

What I actually want from the next cycle is smaller and harder. Settle the question the GDD and the level answered differently, because level design cannot be fixed while the game is two games. Get something playable in front of a person who did not build it inside the first two days, because across five cycles that is the single reliable predictor of whether the build got better. And cut against the scope document when the calendar moves, rather than writing it once and letting it become a description of what we hoped for.

Five prototypes is not enough to be good at this. It has been enough to find out which of my instincts are load bearing and which are just comfortable, and that is a better return than a working build would have been.